Week 3, Day 2: Inter-Service Communication, Feign, gRPC and Sliding Window Maximum

Prepared for Upendra Kumar Sharma | 3 to 4 hours | Main topic: synchronous vs asynchronous communication, REST clients, Feign, gRPC, timeouts | DSA: Queues, stacks and deques

Today's Schedule

- **0:00 to 0:50** Study synchronous vs asynchronous communication and the failure math (sections 1 and 2).
- **0:50 to 1:40** Study REST clients, OpenFeign, timeouts and retries (sections 3 and 4).
- **1:40 to 2:10** Study gRPC and compare the options (section 5).
- **2:10 to 2:30** Study contracts, security and tracing across services (section 6).
- **2:30 to 3:20** DSA: solve the 2 problems in section 8. Spend at most 25 minutes per problem before reading the hint.
- **3:20 to 3:50** Answer the interview questions aloud (section 9) and add 10 flashcards.
- **Optional (60 min):** do the hands-on exercise in section 7 using your Day 1 skeleton.

1. Synchronous vs Asynchronous Communication

Synchronous (request and response)

- The caller sends a request and **waits** for the answer. Protocols: HTTP and REST, gRPC.
- **Good for:** queries that need an answer now (get a price, check stock), and simple flows.
- **Costs:** the caller and callee must both be up at the same moment (**temporal coupling**), latency adds up along a call chain, and a slow service ties up the caller's threads.

Asynchronous (messaging and events)

- The sender puts a message on a broker (Kafka, RabbitMQ, SQS) and **moves on**. Consumers process it later.
- **Good for:** long-running work, fan-out to several consumers, absorbing traffic spikes, and keeping services working when a downstream one is down.
- **Costs:** eventual consistency, harder debugging, and you must handle duplicates, ordering and failures (Day 5 covers Kafka in depth).

Commands vs events

- A **command** says do this and has one intended receiver (ReserveStock).
- An **event** says something happened and can have many subscribers (OrderPlaced). The publisher does not know or care who listens.
- Events give looser coupling. Commands give clearer control.

How to choose

- Does the user need the result in the same request? Use synchronous.
- Does the work take long, or must it survive a downstream outage? Use asynchronous.
- Do many services react to the same fact? Publish an event.
- In practice, systems **mix both**: synchronous for queries at the edge, asynchronous for state changes between services.

2. Why Synchronous Chains Are Risky

- **Availability multiplies.** If a request needs five services in a chain, each 99.9% available, the combined availability is about 0.999 to the power 5, roughly **99.5%**. That is about 3.6 hours of downtime a month instead of 43 minutes.
- **Latency adds up.** Five calls of 100 ms each take at least 500 ms before any processing.
- **Cascading failure:** if service C becomes slow, B's threads pile up waiting for C, then A's threads pile up waiting for B, and the whole system stalls even though only C was sick.
- **Retry storms:** when many callers retry a struggling service at the same time, they can finish it off.
- **Mitigations:** timeouts on every call, circuit breakers and bulkheads (Day 3), fewer hops, caching, and asynchronous flows where possible.

3. REST Clients in Spring

- **RestTemplate:** the classic blocking client. It still works but is in maintenance mode, so prefer newer clients for new code.
- **RestClient (Spring 6.1 and Spring Boot 3.2 and later):** a modern fluent **blocking** client, and a good default for new synchronous code.
- **WebClient:** reactive and non-blocking (needs spring-webflux). Best when you really are reactive, or when you need to fan out many calls concurrently. Calling block() on it works but removes the benefit.
- **HTTP Interface clients (Spring 6):** declare an interface with @GetExchange and similar annotations, and Spring generates the implementation. Similar in spirit to Feign, with no extra dependency.

- **OpenFeign (Spring Cloud OpenFeign):** the declarative client that integrates with service discovery, load balancing and Resilience4j. Very common in microservice interviews.

4. OpenFeign, Timeouts and Retries

A Feign client

```
@SpringBootApplication
@EnableFeignClients
public class OrderServiceApplication { }

@FeignClient(name = "inventory-service", path = "/api/inventory")
public interface InventoryClient {

    @GetMapping("/{sku}")
    StockResponse getStock(@PathVariable("sku") String sku);

    @PostMapping("/reserve")
    ReservationResponse reserve(@RequestBody ReserveRequest request);
}
```

- The name inventory-service is resolved through the service registry and load balanced by Spring Cloud LoadBalancer, exactly like lb:// in the gateway.
- You inject InventoryClient like any other bean and call it like a local method. That convenience hides a **network call that can fail or be slow**, so never treat it as local.

Configuring timeouts (always do this)

- **Connect timeout:** how long to wait to establish the connection.
- **Read (response) timeout:** how long to wait for data once connected.
- Without sensible timeouts, a slow downstream holds your threads for a very long time (Feign's defaults are generous, about 10 seconds to connect and 60 seconds to read).

```
spring:
  cloud:
    openfeign:
      client:
        config:
          default:
            connectTimeout: 2000
            readTimeout: 3000
            loggerLevel: basic
          inventory-service:
            readTimeout: 2000
```

Timeout budgeting

- Timeouts must **shrink as you go deeper**. If the gateway allows 5 seconds, the order service should give each downstream call less, for example 2 seconds, and leave time for its own work and a possible retry.
- A callee that keeps working after its caller has given up wastes resources. Pass a **deadline** or timeout downstream where you can.

Retries

- Retry only **transient** failures (timeouts, connection errors, 503), never client errors such as 400 or 404.
- Retry only **idempotent** operations, or operations protected by an **idempotency key**. Retrying a non-idempotent POST can charge a customer twice.
- Use a **small number of attempts** with **exponential backoff and jitter** (random variation), so retries from many callers do not line up and flood the service.
- Combine retries with a circuit breaker (Day 3). Be careful about **retry amplification**: if each of three layers retries three times, one request can become 27.
- Feign's own Retryer is disabled by default in Spring Cloud. Prefer Resilience4j retry so you control it centrally.

Error handling

- Feign throws **FeignException** (with subclasses such as FeignException.NotFound) for non-2xx responses.
- Provide a custom **ErrorDecoder** to translate responses into your own domain exceptions, so service code does not depend on HTTP details.
- Decide what happens on failure: fail the request, return a default, use cached data, or queue the work for later. That choice is a business decision, not just a technical one.

Propagating context with a RequestInterceptor

```
@Component
public class ContextPropagationInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        String correlationId = MDC.get("correlationId");
        if (correlationId != null) {
            template.header("X-Correlation-Id", correlationId);
        }
        // also forward the Authorization header, or fetch a service token
    }
}
```

5. gRPC and Comparing the Options

What gRPC is

- A high-performance RPC framework from Google. It runs over **HTTP/2** and sends **Protocol Buffers** (a compact binary format) defined in a **.proto file**. Code for the client and server is generated from that schema.
- **Streaming:** unary (one request, one response), server streaming, client streaming, and bidirectional streaming.
- **Strengths:** small payloads, low latency, strongly typed contracts, code generation for many languages, built-in streaming, deadlines and cancellation.
- **Weaknesses:** binary payloads are hard to read and debug, browsers cannot call it directly (you need gRPC-Web or a gateway), load balancing needs HTTP/2-aware infrastructure, and the learning curve is steeper than REST.
- **Best fit:** internal service-to-service traffic with high volume or latency sensitivity, polyglot systems, and streaming.
- **Spring support:** through community starters or the Spring gRPC project. Check which one fits your Spring Boot version.

```
syntax = "proto3";

service InventoryService {
  rpc GetStock (StockRequest) returns (StockReply);
}

message StockRequest { string sku = 1; }
message StockReply   { string sku = 1; int32 available = 2; }
```

- **Schema evolution rules:** never reuse or change a field number, add new fields as optional with new numbers, and never remove a field that old clients still send.

REST vs gRPC vs messaging vs GraphQL

- **REST over JSON:** universal, human readable, easy to cache, great for public and browser-facing APIs. Larger payloads and no built-in contract enforcement.
- **gRPC:** best raw performance and contracts for internal calls.
- **Messaging:** best for decoupling and resilience.
- **GraphQL:** lets clients choose exactly the fields they need, useful for varied front ends, but it moves complexity to the server (N+1 on resolvers, caching, query cost limits).

6. Contracts, Security and Tracing Across Services

- **API contracts and compatibility:** change APIs in **backward-compatible** ways (add fields, do not rename or remove). Consumers should be **tolerant readers** that ignore fields they do not know.
- **Consumer-driven contract tests** (Pact or Spring Cloud Contract) let each consumer record what it needs, and the provider's build fails if a change would break it. They catch integration breaks without running the whole system.
- **Avoid chatty calls:** one request that triggers dozens of small calls is slow and fragile. Offer batch endpoints, return what callers need, or use an aggregation layer.
- **Service-to-service authentication:**
  - Forward the **user's JWT** so downstream services can authorize on behalf of the user, or
  - Use **OAuth2 client credentials** so one service authenticates as itself, or
  - Use **mutual TLS** (often provided by a service mesh) for transport-level identity.
  - Services must still authorize requests. Being on the internal network is not enough.
- **Correlation and tracing:** generate a correlation id (or W3C trace context) at the edge and propagate it on every call and message. With Micrometer Tracing the trace and span ids also appear in your logs, so you can follow one request across services.
- **Fallback and degradation:** decide what the user sees when a dependency is down, such as cached prices, a reduced feature set, or a message that work will complete later.

7. Optional Hands-On (60 Minutes)

Using the Day 1 skeleton (discovery server, config server, gateway, order-service):

1. Create or reuse **inventory-service** with GET /api/inventory/{sku}, registered in Eureka.
2. In order-service, add the Feign client above and call inventory when creating an order.

3. Set the Feign read timeout to 2 seconds. In inventory-service add a controller flag that sleeps for 5 seconds. Call it and watch the **timeout** and the exception.
4. Add a custom ErrorDecoder that maps 404 to a StockNotFoundException, and handle it in your @RestControllerAdvice.
5. Add the correlation id interceptor and a servlet filter that creates the id. Confirm the same id shows up in both services' logs.
6. Run several parallel requests while inventory sleeps, and check how many request threads in order-service are blocked. This is the cascading failure problem, and tomorrow's circuit breaker fixes it.

Keep notes of what you saw, because it makes a strong answer to "how do you handle a slow downstream service".

8. DSA: Queues, Stacks and Deques (2 Problems)

Problem 1: Implement Queue using Stacks

- **Idea:** use two stacks. Push new items onto the **in** stack. For pop or peek, if the **out** stack is empty, move **all** items from in to out (this reverses their order), then use out. Move only when out is empty, otherwise you break the order.
- **Complexity:** push is O(1). pop and peek are **amortized O(1)**, because each element is moved at most once. A single pop can take O(n).
- **Edge cases:** pop on an empty queue, interleaving pushes and pops, peek after pops.
- **Variation to prepare:** Implement Stack using Queues.

```
class MyQueue {
    private final Deque<Integer> in = new ArrayDeque<>();
    private final Deque<Integer> out = new ArrayDeque<>();

    public void push(int x) { in.push(x); }

    public int pop() {
        shift();
        return out.pop();
    }

    public int peek() {
        shift();
        return out.peek();
    }

    public boolean empty() { return in.isEmpty() && out.isEmpty(); }
```

```
    private void shift() {
        if (out.isEmpty()) {
            while (!in.isEmpty()) out.push(in.pop());
        }
    }
}
```

Problem 2: Sliding Window Maximum

- **Idea:** keep a **monotonic decreasing deque of indices**. For each new element, remove indices from the front that have left the window, remove indices from the back whose values are not larger than the new value (they can never be a maximum again), then add the new index. The front of the deque is always the index of the current maximum.
- **Complexity:** O(n) time, because each index is added and removed at most once. O(k) space.
- **Edge cases:** k equal to 1, k equal to the array length, a strictly decreasing array, duplicate values (use less than or equal when popping from the back, or handle equal values consistently).
- **Alternatives:** a max-heap of values and indices with lazy deletion is O(n log n), and a brute-force scan is O(n x k).
- **Pattern to remember:** the same monotonic idea as Daily Temperatures, extended to a window with a deque.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    if (n == 0) return new int[0];
    int[] result = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();            // indices, values
decreasing
    for (int i = 0; i < n; i++) {
        while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
        while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i])
dq.pollLast();
        dq.offerLast(i);
        if (i >= k - 1) result[i - k + 1] = nums[dq.peekFirst()];
    }
    return result;
}
```

9. Interview Questions to Answer Aloud

1. Synchronous vs asynchronous communication between services: how do you choose?
2. What is temporal coupling, and why does it hurt availability?
3. Five services are each 99.9% available and called in a chain. What is the overall availability?
4. What is cascading failure, and how do timeouts and bulkheads help?
5. RestTemplate vs RestClient vs WebClient vs Feign: when would you use each?
6. What are connect and read timeouts, and how would you set timeouts across a chain of three services?
7. When is it safe to retry a request? What are backoff and jitter, and what is retry amplification?
8. How do you make a POST safe to retry?
9. How do you translate downstream HTTP errors into your own exceptions in Feign?
10. gRPC vs REST: what are the differences, and when would you pick gRPC?
11. How do you evolve a Protocol Buffers schema or a REST API without breaking consumers?
12. What are consumer-driven contract tests, and what problem do they solve?
13. How do you authenticate service-to-service calls, and how do you trace one request across services?

10. End-of-Day Checklist

- [ ] I can explain when to use synchronous and asynchronous communication, and give the commands vs events distinction.
- [ ] I can explain cascading failure and the availability math.
- [ ] I can write a Feign client, configure its timeouts, and add an error decoder and interceptor.
- [ ] I can state the rules for safe retries and timeout budgets.
- [ ] I can compare REST, gRPC, messaging and GraphQL.
- [ ] I solved both DSA problems and can explain the two-stack queue and the monotonic deque.
- [ ] (Optional) I reproduced a slow downstream call and saw the timeout and thread blocking.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 3, Day 3): Resilience4j (circuit breaker, retry, bulkhead, rate limiter, time limiter, fallbacks), plus binary tree problems.